EXERCISE – IMPLEMENTING ARTIFICIAL NEURAL NETWORKS (ANN)

AIM

To implement a simple Artificial Neural Network (ANN) using Python (Keras/TensorFlow) to classify handwritten digits from the MNIST dataset and evaluate its performance.

PROCEDURE

1. Import the required libraries: TensorFlow/Keras for the neural network, and utilities for data loading and preprocessing.
2. Load the MNIST dataset of handwritten digits (60,000 training images and 10,000 test images).
3. Normalize the pixel values of the images to the range [0, 1] by dividing by 255.
4. Convert the class labels (0–9) into one-hot encoded vectors.
5. Define a Sequential neural network model with a Flatten layer, a Dense hidden layer (128 neurons, ReLU activation), and a Dense output layer (10 neurons, Softmax activation).
6. Compile the model using the Adam optimizer, categorical cross-entropy loss, and accuracy as the metric.
7. Train the model on the training data for a specified number of epochs (e.g., 5) with a validation split.
8. Evaluate the trained model on the test set and print the test accuracy.
9. Run the program and observe the training progress and final accuracy.

PROGRAM

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical

# Load data
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

# One-hot encode labels
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

# Define ANN
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile & Train
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5, validation_split=0.1)

# Evaluate
loss, acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", acc)
```

OUTPUT

```
Downloading data from
https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ■■■■■■■■■■■■■■■■■■■■■■ 1s 0us/step

Epoch 1/5
```

```
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.8642 - loss: 0.4699 -
val_accuracy: 0.9648 - val_loss: 0.1215
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9611 - loss: 0.1288 -
val_accuracy: 0.9690 - val_loss: 0.1027
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.9755 - loss: 0.0857 -
val_accuracy: 0.9747 - val_loss: 0.0867
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9822 - loss: 0.0614 -
val_accuracy: 0.9755 - val_loss: 0.0840
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9871 - loss: 0.0443 -
val_accuracy: 0.9765 - val_loss: 0.0797

313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9730 - loss: 0.0902
Test Accuracy: 0.9757000207901001
```

Result: The Artificial Neural Network was successfully implemented and trained on the MNIST dataset. The model achieved a test accuracy of approximately 97.57%, demonstrating effective classification of handwritten digits.